Documentación Técnica

Sistema de Reservas y Boletos Aéreos sobre Apache Cassandra (aerolinea_db)

Nombre: Luis Pedro Pérez Monzón

Carné: 201901271

---

1. Introducción

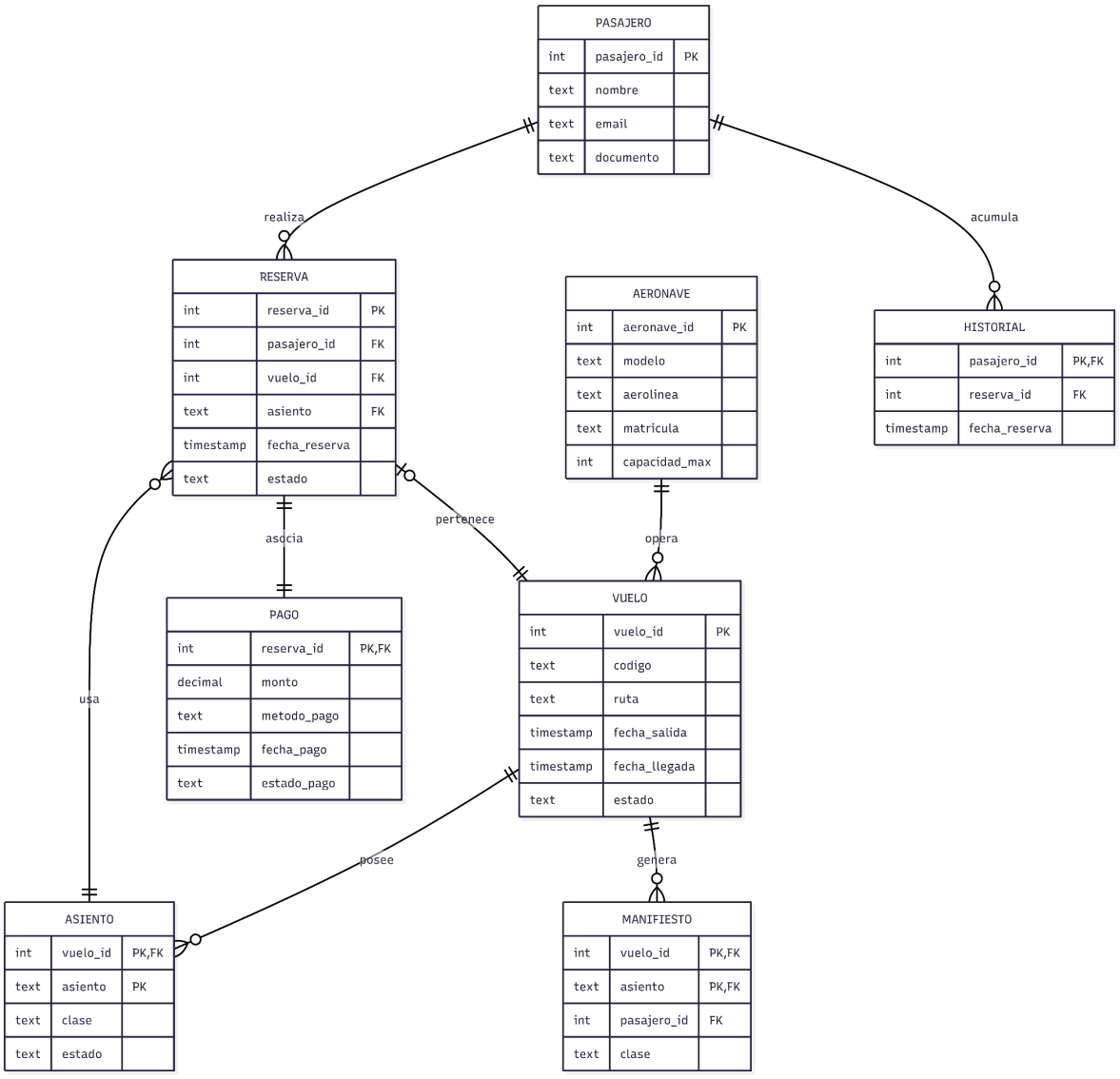
Se implementó la base de datos distribuida de una aerolínea en Apache Cassandra (clúster de 3 nodos en Docker), siguiendo el enfoque Query-Driven Modeling: cada tabla está diseñada para resolver una sola consulta de negocio con una lectura de partición, sin `JOIN`, sin `ALLOW FILTERING` y sin `COUNT` sobre tablas base en tiempo de lectura (las agregaciones viven en la BD).
Datos cargados y verificados:

Indicador	**Valor**
Reservas (batch)	164,234
Pagos por reserva	164,234
Historial de pasajeros	164,234
Manifiesto de vuelo (detalle)	151,244
Q1 — conteo de asientos por (vuelo, clase)	3,600
Q4 — ocupación por (ruta, mes, vuelo)	1,200
Q5 — ingresos por (mes, vuelo)	3,318
Nodos	3 (`cassandra1/2/3`)
Replication factor	3
Consistency operativo	QUORUM
Estrategia de replicación	NetworkTopologyStrategy (dc1)

2. Modelo conceptual (entidad-relación)

Entidades: Aeronave, Vuelo, Asiento, Pasajero, Reserva, Pago e Historial, con las reglas:

- Una aeronave opera muchos vuelos (1 : N).
- Un vuelo posee muchos asientos, cada uno con clase y estado (1 : N).
- Un pasajero realiza muchas reservas (1 : N).
- Cada reserva pertenece a un vuelo y a un asiento (N : 1), y se asocia a un pago (1 : 1).
- Cada cambio de estado deja un registro en el historial (1 : N).
- Un asiento de un vuelo es exclusivo para un pasajero a la vez.



Entidad	Atributos
Aeronave	aeronave_id, modelo, aerolinea, matricula, capacidad_max
Pasajero	pasajero_id, nombre, email, documento, telefono, nacionalidad
Vuelo	vuelo_id, codigo_vuelo, aeronave_id, vuelo, origen, destino, ruta, fecha_salida, fecha_llegada, estado
Asiento	asiento, clase, estado
Reserva	reserva_id, pasajero_id, vuelo_id, asiento, fecha_reserva, estado
Pago	reserva_id, monto, metodo_pago, fecha_pago, estado_pago
Historial	pasajero_id, fecha_reserva, reserva_id, vuelo_id, estado_reserva, estado_pago, ...

---

3. Fundamentos del diseño en Cassandra

En Cassandra no existen los JOINs y `ALLOW FILTERING` es un anti-patrón. Por ello se aplica el modelado orientado a consultas: "una tabla por patrón de acceso". La clave se elige para que la consulta complete en una sola partición, y las columnas de clustering proveen orden y rangos nativos. Las consultas del negocio (Q1–Q5) se definieron primero; luego se diseñó una tabla para cada una.

3.1 Keyspace y replicación

```
CREATE KEYSPACE aerolinea_db
WITH replication = {
  'class'              : 'NetworkTopologyStrategy',
  'dc1'                : 3
};
```

- NetworkTopologyStrategy (recomendada por DataStax; soporta múltiples datacenters y RF por DC). En producción no se usa `SimpleStrategy`.

- RF = 3: cada fila se replica en los 3 nodos del DC; permite QUORUM (2 de 3) y tolera la caída de un nodo.

---

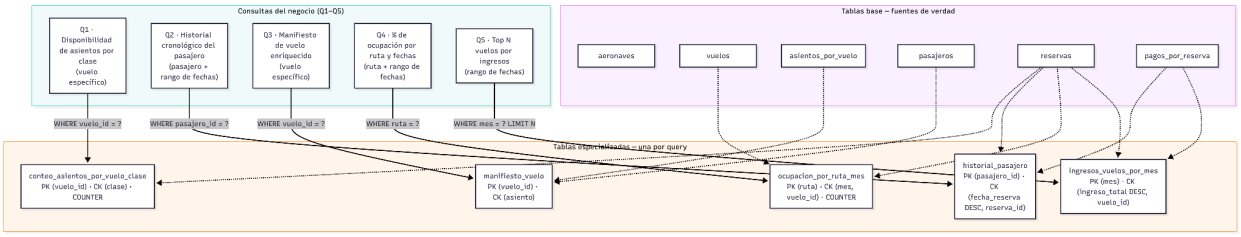
4. Modelo lógico (query-driven) y CQL

Tablas base (fuentes de verdad) y especializadas (una por consulta):

Tabla	**Propósito**	**Clave de partición**	**Clustering**
`aeronaves`	Parque aéreo	aeronave_id	—
`vuelos`	Programación (denormaliza ruta, capacidad)	vuelo_id	—
`asientos_por_vuelo`	Asientos de cada vuelo	vuelo_id	asiento
`pasajeros`	Directorio	pasajero_id	—
`reservas`	Fuente de verdad de reservas	reserva_id	—
`pagos_por_reserva`	Pago de cada reserva	reserva_id	—
`historial_pasajero`	Q2 — historial por pasajero	pasajero_id	(fecha_reserva DESC, reserva_id)
`manifiesto_vuelo`	Q3 — manifiesto del vuelo	vuelo_id	asiento
`conteo_asientos_por_vuelo_clase`	Q1 — COUNTER	vuelo_id	clase
`ocupacion_por_ruta_mes`	Q4 — COUNTER	ruta	(mes, vuelo_id)
`ingresos_vuelos_por_mes`	Q5 — DETALLE top	mes	(ingreso_total DESC, vuelo_id)

4.1 Mapa query-driven (consultas → tablas)

El siguiente diagrama resume el enfoque query-driven: cada consulta del negocio (Q1–Q5) se resuelve con una única tabla especializada cuya clave de partición coincide con el `WHERE` de la query, y cuyas clustering keys proveen orden y rangos nativos. Las tablas especializadas se denormalizan desde las tablas base (indicado con flechas discontinuas) para eliminar JOINs:



4.2 Tabla base `reservas` (fuente de verdad)

```
CREATE TABLE aerolinea_db.reservas (
    reserva_id      int PRIMARY KEY,
    pasajero_id     int,
    vuelo_id        int,
    asiento         text,
    fecha_reserva   timestamp,
    estado          text
);
```

4.3 Tabla base `pagos_por_reserva` + `historial_pasajero` (Q2)

Cada reserva tiene **un pago** y se denormaliza su **historial** por pasajero para responder Q2 (historial cronológico del pasajero con estado de reserva y de pago) en **una sola partición**:

```
CREATE TABLE aerolinea_db.historial_pasajero (
    pasajero_id     int,
    fecha_reserva   timestamp,
    reserva_id      int,
    vuelo_id        int,
    codigo_vuelo    text,
    origen          text,
    destino         text,
    fecha_salida    timestamp,
    asiento         text,
    clase           text,
    estado_reserva  text,
```

```
    estado_pago     text,
    monto           decimal,
    metodo_pago     text,
    fecha_pago      timestamp,
    PRIMARY KEY (pasajero_id, fecha_reserva, reserva_id)
) WITH CLUSTERING ORDER BY (fecha_reserva DESC, reserva_id DESC);
```

- Partition key = pasajero_id → todo el historial del pasajero es una partición; consulta por rango de fechas = rango de clustering sin `ALLOW FILTERING`.
- Clustering (fecha_reserva, reserva_id) DESC → orden cronológico inverso nativo.

4.4 Tabla base `manifiesto_vuelo` (Q3)

Manifiesto del vuelo (asiento, pasajero, documento, clase, estados): partition key `vuelo_id`, clustering `asiento` (ordena el manifiesto por número de asiento).

---

5. Tablas especializadas (agregación por consulta)

5.1 Q1 — `conteo_asientos_por_vuelo_clase` (COUNTER)

```
CREATE TABLE aerolinea_db.conteo_asientos_por_vuelo_clase (
    vuelo_id     int,
    clase        text,
    disponibles  counter,
    ocupados     counter,
    bloqueados   counter,
    PRIMARY KEY (vuelo_id, clase)
);
```

- Partition key = vuelo_id → todas las clases del vuelo en una partición → Q1 (`WHERE vuelo_id = ?`) es una sola lectura.
- Clustering por `clase` para ordenar el desglose.
- COUNTER: el conteo cambia con cada reserva/cancelación; en lugar de recontar 151,244 asientos en lectura, se incrementa con `UPDATE ... SET disponibles = disponibles + ?`.

5.2 Q4 — `ocupacion_por_ruta_mes` (COUNTER)

```
CREATE TABLE aerolinea_db.ocupacion_por_ruta_mes (
    ruta                 text,
    mes                  text,    -- "YYYY-MM"
    vuelo_id             int,
    reservas_confirmadas counter,
    capacidad            counter,
    PRIMARY KEY (ruta, mes, vuelo_id)
) WITH CLUSTERING ORDER BY (mes ASC);
```

- Partition key = ruta → todos los vuelos de la ruta en una partición.
- Clustering `(mes, vuelo_id)` permite rangos de meses; la ocupación % = `reservas_confirmadas / capacidad` con dos counters de la misma fila.

5.3 Q5 — `ingresos_vuelos_por_mes` (DETALLE, clustering DESC)

```
CREATE TABLE aerolinea_db.ingresos_vuelos_por_mes (
    mes            text,
    ingreso_total  decimal,
    vuelo_id       int,
    codigo_vuelo   text,
    PRIMARY KEY (mes, ingreso_total, vuelo_id)
) WITH CLUSTERING ORDER BY (ingreso_total DESC, vuelo_id ASC);
```

- Partition key = mes; clustering `ingreso_total DESC` → el Top-N ya está ordenado físicamente: `SELECT ... WHERE mes = ? LIMIT 10` devuelve el Top-10 sin ORDER BY en el cliente.
- Es una tabla de DETALLE (no COUNTER, porque `ingreso_total` es clave de clustering): se reconstruye con `TRUNCATE + INSERT` agregado desde los pagos confirmados, idéntico al proceso de carga (`load_data.py`) y idempotente.

---

6. Configuración del clúster y consistencia

Parámetro	**Valor**	**Justificación**
Nodos	3 (cassandra1/2/3)	Mínimo para RF=3 y QUORUM

Parámetro	Valor	Justificación
Replication	NetworkTopologyStrategy, RF=3	DC-aware; réplica completa
Consistency base	QUORUM	Equilibrio consistencia/disponibilidad; tolera 1 nodo caído
Consistency de escritura	QUORUM	Confirma en 2 de 3 réplicas
puertos CQL	9042 / 9142 / 9242	Un puerto por nodo

---

7. Consultas CQL implementadas

Las 5 consultas Q1–Q5 se resuelven las tablas especializadas. Ver ejemplos completos y parámetros en el Manual de Usuario (`Manual_Usuario.md`). Todas operan sin ALLOW FILTERING (el filtro es siempre la clave de partición, y los rangos son clustering keys).

---

8. Pruebas de tolerancia a fallos (métricas por Consistency Level)

Se simuló la caída del nodo 2 (`cassandra2`, puerto 9142) y se midió, con 2 de 3 nodos vivos, la misma consulta con diferentes Consistency Levels (10 intentos cada una):

Consistency Level	Intentos	Éxitos	Fallos	Disponibilidad
`ONE`	10	10	0	100 %
`QUORUM`	10	10	0	100 %
`ALL`	10	0	10	0 % — `UnavailableException`: *required 3, alive 2*

Interpretación: con RF=3 y CL=QUORUM el clúster tolera la caída de un nodo manteniendo lecturas correctas y disponibles. `ALL` queda indisponible ante la caída de una sola réplica (esperado), y `ONE` prioriza disponibilidad con menor garantía de consistencia. Latencia comparativa (clúster sano): `ONE` < `QUORUM` < `ALL`; QUORUM añade el round-trip de 2 réplicas.

Recuperación

Al reiniciar `cassandra2`, `nodetool status` lo reincorpora como UN (Up/Normal) en el anillo; la replicación con RF=3 y el mecanismo de hinted handoff re-sincronizan los datos sin intervención manual — el clúster recupera consistencia total automáticamente.

---

9. Verificación y reparación (idempotente)

`verificar_reparar.py`:

- Cuenta filas por tabla y compara contra lo esperado.
- Reconstruye las tablas de agregación Q1/Q4/Q5 (TRUNCATE + INSERT/UPDATE) de forma idempotente (repetir no duplica).
- Corrige faltantes en tablas base re-insertando desde los CSV.
- Es auditado por conteos finales (3,600 / 1,200 / 3,318).

---

10. Conclusiones

1. El modelado por consulta con tablas denormalizadas responde Q1–Q5 sobre 164,234 reservas en milisegundos, sin JOINs ni ALLOW FILTERING.
2. La clave de partición adecuada (coincide con el `WHERE`) es el factor decisivo: ubica los datos de la consulta en una sola réplica.
3. RF=3 + NetworkTopologyStrategy + CL=QUORUM da el mejor balance de consistencia y disponibilidad y tolera la caída de un nodo, como se verificó en las pruebas de fallo.
4. Las COUNTER (Q1, Q4) trasladan la agregación a la escritura y las tablas de DETALLE (Q5) usan clustering DESC para el Top-N sin ordenar en el cliente.